Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 6

Inside the xv6 File System

Track B: Build Your Own Small OS (xv6)

Elaborated by:
st. gr. FAF-243
Veacheslav Morari

Verified by:
Patricia Reţman

Chişinău – 2026

Contents

Introduction **2**

1 Read what xv6 shows you **2**

2 Write mystat **2**
2.1 Hard links at work 3

3 Measure the limits **4**
3.1 Block size and address limits 4
3.2 The name limit 4
3.3 The size of the file system image 4

4 Write find **4**
4.1 Searching nested directories 6
4.2 Searching from a specific directory 6
4.3 The missing-argument case 6
4.4 Searching for a missing name 7
4.5 Searching for a directory and for a system file 7

Conclusion **7**

Introduction

The purpose of this laboratory work is to study the file system of xv6 from the inside: the directory structure, inodes, hard links, the fixed limits that follow from `kernel/fs.h`, and the recursive traversal of the directory tree. Two new programs are added, `mystat`, which prints the metadata of a file, and `find`, which walks the tree and prints every path whose last component matches a given name. The ideas are the same as in ext4 and other Unix file systems, but xv6 shows them in a few hundred readable lines.

1. Read what xv6 shows you

The file `kernel/stat.h` defines three types used by the file system: `T_DIR = 1` for a directory, `T_FILE = 2` for a regular file, and `T_DEVICE = 3` for a device. The root directory has inode 1. Its entries `.` and `..` point to the same inode, because the parent of the root directory is the root directory itself.

An empty directory already has a size greater than zero, because it contains the two entries `.` and `..`. On the fresh xv6 disk a newly created directory `a` has a size of 32 bytes. The structure `struct dirent` is 16 bytes long: 2 bytes for `inum` and 14 bytes for `name`. Two entries occupy 32 bytes.

```
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

The exact inode number of the new directory depends on the state of the file system image.

2. Write mystat

The program `user/mystat.c` opens every path given on the command line, calls `stat()` to fill a `struct stat`, decides the type of the object with a small helper function, and prints the name, the type, the inode number, the number of links and the size in bytes. On error it prints a message to standard error and goes on with the next argument.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
    if(t == T_DIR)    return "directory";
    if(t == T_FILE)   return "file";
    if(t == T_DEVICE) return "device";
    return "?";
}

int
```

```
main(int argc, char *argv[])
{
    struct stat st;
    for(int i = 1; i < argc; i++){
        if(stat(argv[i], &st) < 0){
            fprintf(2, "mystat: cannot stat %s\n", argv[i]);
            continue;
        }
        printf("%s: %s, inode %d, links %d, size %d bytes\n",
               argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
    }
    exit(0);
}
```

After the line `$U/_mystat\` was added to `UPROGS` and xv6 was rebuilt, the program produced the expected output:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
```

2.1. Hard links at work

The hard-link experiment confirmed what a hard link really is.

```
$ mystat README
README: file, inode 2, links 1, size 2441 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
```

After `ln README r2` both names showed inode 2 and the link count became 2, which means that the two names point to the same inode. After `rm README` the file was still readable under `r2`, and the link count went back to 1.

A hard link is simply an additional name of the same inode. The command `rm` removes the name from the directory and decrements the counter. The data are freed only when the counter reaches zero and no process keeps the file open.

In `kernel/sysfile.c`, the function `sys_unlink()` performs the decrement with the single line `ip->nlink-;`.

In `kernel/fs.h`, the structure `struct dinode` holds the fields `type`, `major`, `minor`, `nlink`, `size` and the array of block addresses `addrs`. xv6 has no symbolic links and no standard Unix permission bits.

3. Measure the limits

3.1. Block size and address limits

The following command printed the four defines that fix the geometry of the file system:

```
$ grep -nE 'define BSIZE|define NDIRECT|define NINDIRECT|define DIRSIZ' kernel/
    fs.h
#define BSIZE   1024
#define NDIRECT 12
#define NINDIRECT (BSIZE / sizeof(uint))
#define DIRSIZ  14
```

The block size is 1024 bytes. An inode keeps 12 direct block pointers and one indirect block, which itself holds 256 block addresses of 4 bytes each. The maximum number of data blocks of one file is therefore $12 + 256 = 268$ blocks, and the maximum size is $268 \times 1024 = 274\,432$ bytes, i.e. 268 KiB.

3.2. The name limit

A directory with a long name was created to see what xv6 shows:

```
$ mkdir averyveryverylongname
$ ls
...
averyveryveryl 1 27 32
...
```

The name was cut after 14 characters, because `DIRSIZ = 14`. xv6 stores no more than 14 characters of a name in a directory entry; longer names are silently truncated.

3.3. The size of the file system image

While `make` builds `fs.img`, the tool `mkfs` prints the layout of the disk:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks
    1953 total 2000
```

The image contains 2000 blocks in total. Of them, 47 blocks are metadata: the boot block, the superblock, the log (31 blocks), the inode blocks (13) and the bitmap (1). The remaining 1953 blocks form the data area. The free space is a little smaller than the data area, because some blocks are already used by the files and directories that are copied in by `mkfs`.

4. Write find

The program `user/find.c` walks the directory tree recursively and prints every path whose last component matches the second argument. It starts from `user/ls.c`, which already shows how to open a directory, read `struct dirent` entries one by one, build the

full path in a buffer and call `stat` on it. The free slots (`inum == 0`) and the entries `.` and `..` are skipped, and every opened file descriptor is closed before the function returns.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

void
find(char *path, char *name)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, 0)) < 0){
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }
    if(fstat(fd, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", path);
        close(fd);
        return;
    }

    switch(st.type){
    case T_DEVICE:
    case T_FILE:
        p = path + strlen(path);
        while(p > path && *(p - 1) != '/')
            p--;
        if(strcmp(p, name) == 0)
            printf("%s\n", path);
        break;

    case T_DIR:
        if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
            printf("find: path too long\n");
            break;
        }
        strcpy(buf, path);
        p = buf + strlen(buf);
        *p++ = '/';
        while(read(fd, &de, sizeof(de)) == sizeof(de)){
            if(de.inum == 0)
                continue;
            memmove(p, de.name, DIRSIZ);
            p[DIRSIZ] = 0;
            if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
                continue;
            if(stat(buf, &st) < 0){
```

```
                fprintf(2, "find: cannot stat %s\n", buf);
                continue;
            }
            if(strcmp(p, name) == 0)
                printf("%s\n", buf);
            if(st.type == T_DIR)
                find(buf, name);
        }
        break;
    }
    close(fd);
}

int
main(int argc, char *argv[])
{
    if(argc < 3){
        fprintf(2, "Usage: find <dir> <name>\n");
        exit(1);
    }
    find(argv[1], argv[2]);
    exit(0);
}
```

4.1. Searching nested directories

```
$ mkdir a
$ mkdir a/b
$ echo x > a/r2
$ echo y > a/b/r2
$ find . r2
./a/b/r2
./a/r2
```

The order of the lines may differ, because it follows the order of the entries in the directory.

4.2. Searching from a specific directory

```
$ find a b
a/b
```

4.3. The missing-argument case

```
$ find
Usage: find <dir> <name>
```

4.4. Searching for a missing name

```
$ find . nonexistent
$
```

No match is printed and the program exits silently.

4.5. Searching for a directory and for a system file

```
$ find . a
./a
$ find . README
./README
```

Conclusion

In this work the file system of xv6 was studied from inside. The directory structure, inodes, hard links and the limits of the file system were observed by experiment. The program `mystat` prints the metadata of a file in a readable form and made the hard-link experiment easy to verify: two names of the same file shared one inode, the link counter followed `ln` and `rm`, and the data were freed only when the counter reached zero. The reading of `kernel/fs.h` and the output of `mkfs` gave the fixed numbers of the file system: block size 1024 bytes, 12 direct and 256 indirect pointers, 14 characters for a name, and a 2000-block image divided into 47 metadata blocks and 1953 data blocks. The maximum file size is 274 432 bytes. Finally, the program `find` walked the tree recursively and printed every path with the given name, which showed how a few primitives (opening a directory, reading `struct dirent`, building a path and calling `stat`) are enough to implement a classic Unix utility.